

Министерство науки и высшего образования Российской Федерации

Федеральное государственное автономное образовательное
учреждение высшего образования
Национальный исследовательский Нижегородский государственный
университет им. Н.И. Лобачевского

Институт информационных технологий, математики и механики

Отчет по лабораторной работе

«Реализация метода Гаусса-Жордана для решения систем линейных алгебраических уравнений»

Выполнил:
студент группы
3825Б1ПМиИ1
Казарин А.М.

Проверила:
Бусько П.В.

Нижний Новгород
2026

Содержание:

Введение	3
Постановка задачи	4
Руководство пользователя	5
Описание программной реализации	6
Результаты экспериментов	8
Заключение	11
Литература	12
Приложение	13

Введение

Эта работа посвящена реализации метода Гаусса-Жордана для решения СЛАУ (Систем Линейных Алгебраических Уравнений). Такую систему можно записать в виде $Ax=b$, где A - матрица коэффициентов, x - искомый вектор состоящий из неизвестных, b - вектор значений в правой части. Так как матрицы представляются через вектора, то выстраивается иерархия, которую можно реализовать в программе, где классы матрицы и решения СЛАУ будут наследоваться от класса вектор. Сам метод решения СЛАУ будет заключаться в выборе ведущего элемента в строке, и проведения элементарных преобразований в остальных строках, чтобы в конечном итоге привести матрицу слева к единичному виду, а вектор значений в правой части, будет как раз будет составлять искомый вектор.

Постановка задачи

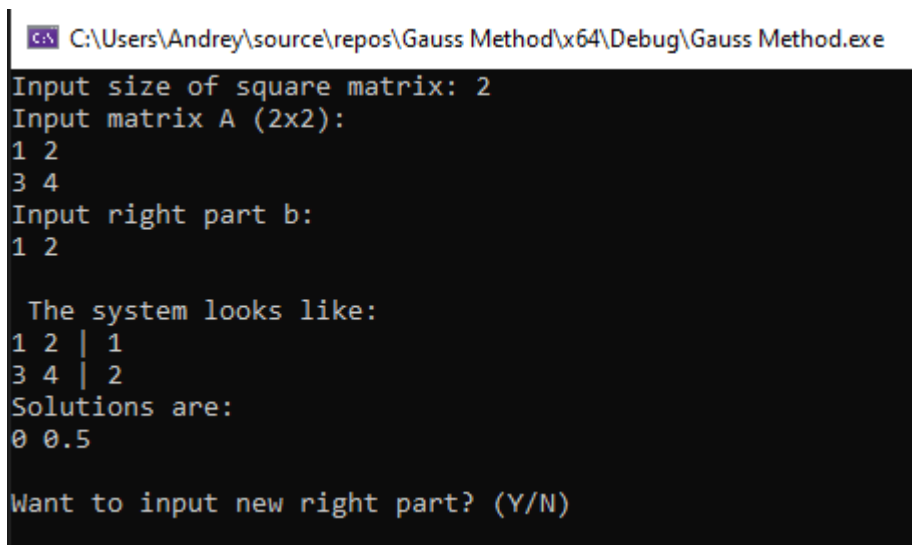
Цель работы: написать программу на языке C++ для решения СЛАУ.

Основные задачи:

1. Использовать шаблоны, чтобы формулы были универсальными для разных типов данных
2. Реализовать принцип наследования от класса вектора, класс матриц и решения матричных уравнений.
3. С помощью исключений обработать различные случаи, когда может быть бесконечное число решений или их не быть вовсе.

Руководство пользователя

Пользователь должен запустить программу, и он увидит в интерфейсе предложение ввести размер матрицы. После этого пользователь сам построчно вводит значения, которые будут входить в матрицу коэффициентов. Далее пользователь сможет ввести значения для вектора левой части. После этого пользователь увидит на экране как выглядит его матрица с отделенной черточками правой частью. И чуть ниже будет представлено решение, где каждой значение искомого вектора будет написано через пробел. Если выяснится, что есть столбец состоящий исключительно из нулей, то программа выдаст, что нет решений, если хотя бы один элемент в правой части не 0. В противном случае будет бесконечное число решений. (Скриншоты с данными результатами будут представлены в разделе “Результаты экспериментов”). Затем для пользователя будет представлена возможность, ещё раз решить СЛАУ, но уже ввести другую правую часть. Для этого пользователь должен ввести Y, на клавиатуре. В случае если пользователь ввёл N, программа завершает работу. (Рис. 1)



```
C:\Users\Andrey\source\repos\Gauss Method\x64\Debug\Gauss Method.exe
Input size of square matrix: 2
Input matrix A (2x2):
1 2
3 4
Input right part b:
1 2

The system looks like:
1 2 | 1
3 4 | 2
Solutions are:
0 0.5

Want to input new right part? (Y/N)
```

Рис. 1 Пример работы программы.

Описание программной реализации

В моём проекте 1 основной файл: Gauss Method.cpp .— это основной файл, в нём содержатся только директивы и функция `main ( )` (Приложение 1). В этом блоке пользователем инициализируется матрица и вектор правой части, который задаёт СЛАУ. Далее используется цикл `do {} while ( )` чтобы имелась возможность изменять правую часть, но матрицу оставлять той же. также внутри происходит обработка исключений внутри блоков `try {}` и `catch ( ) {}`.

Перед мейном, в файле содержится следующий список шаблонных классов и функций, реализованных внутри этих классов:

`class Vector` - основной класс, который будет наследоваться в дальнейшем. Внутри задаётся массив, и размер этого массива в поле `protected`, чтобы была возможность наследовать эти поля. В поле `public` находятся:

`MyVector( ) : mas(nullptr), size(0)` - Стандартный конструктор

`MyVector(int n): size(n)` - Конструктор, внутри которого заполняется массив из `n` элементов

`MyVector(const MyVector& other) : size(other.size)` - Конструктор копирования

`MyVector& operator =(const MyVector& other)` - Перегрузка оператора присваивания

`~MyVector( )` - Деструктор

`int Getsize( )` - Функция для получения размера

`void input( )` - Функция для ввода вектора

`void print( )` - Функция для вывода вектора

`class Matrix : public MyVector<MyVector<T>>` - класс наследник вектора векторов, где строки матрицы будут являться непосредственно векторами. Внутри `protected` задается только размер матрицы. В поле `public`:

`Matrix (int size) : MyVector < MyVector<T>>(size), size_m(size)` - Конструктор для создания матрицы размера `size x size`

`void swap (int r1, int r2)` - Функция для смены строк внутри матрицы

`void input_m( )` - Функция для ввода матрицы

`void print_m( )` - Функция для вывода матрицы

`class SLAU : public Matrix<T>` - Класс, в котором осуществляется задание, решение и вывод СЛАУ. В секции `private` написана функция `bool isZero(T val)`, которая позволяет определить является число нулем или не является. В секции `public`:

`SLAU(int size) : Matrix<T>(size)` - Конструктор для задания СЛАУ

`void printWhole(MyVector<T>& b)` - Вывод на экран, как визуально выглядит матрица с добавленной правой частью, с разделителями

`MyVector<T> Gauss_Jordan(MyVector<T> b)` - Основные пункты работы, такие:

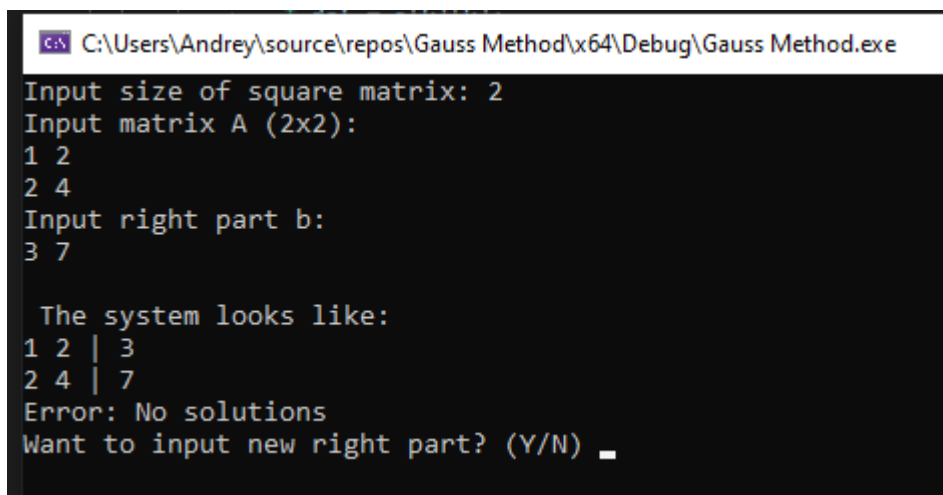
- 1) Создаются копии матрицы и правой части, чтобы не портить оригиналы.
- 2) Ищем строку где находится самый большой элемент по модулю, в `k` столбце. Запоминаем строку и элемент.
- 3) Затем если максимальный элемент в столбце 0, значит этот столбец полностью состоит из нулей и нужно оценить правую часть, чтобы в дальнейшем выбросить исключение, которое будет обработано уже в `main()`
- 4) Меняем строки местами при необходимости, чтобы самый большой по модулю элемент стоял на главной диагонали, он будет ведущим.
- 5) Затем делим строку на главный элемент, чтобы получилась 1 на месте ведущего элемента.
- 6) Вычитаем из всех остальных строк ведущую строку, умноженную на элемент текущей строки в столбце `k`. Так зануляются все элементы в столбце `k`.

В итоге возвращаем правую часть, она и будет ответом на поставленную задачу.

Результаты экспериментов

В качестве экспериментов рассмотрим несколько примеров:

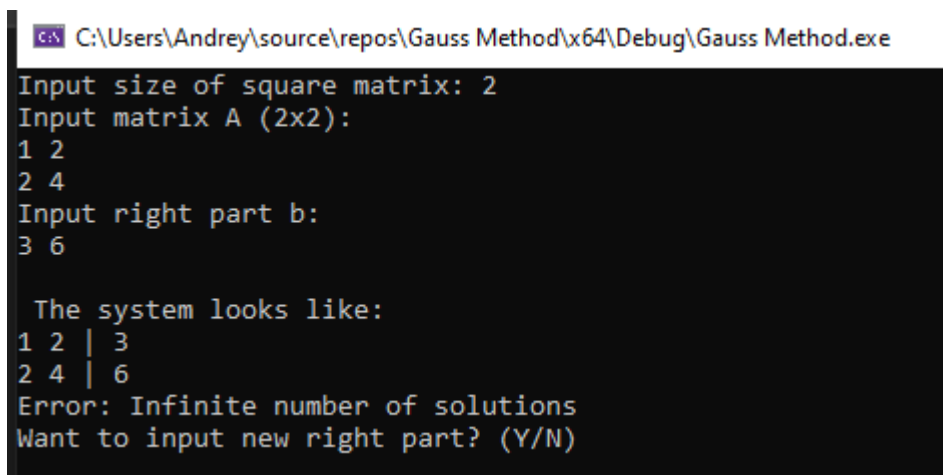
Пример 1 показан, как нормальная работа программы в блоке “Руководство пользователя” (Рис. 1). Результат работы программы совпадает с ответом на эту задачу. Во 2 примере рассмотрим систему с линейно зависимыми векторами (Рис. 2). Результат программы говорит нам, что решений нету (“No solutions”), так и есть. Если поделить вторую строку на 2, то будет видно, что для одного и того же вектора, значения правой части разные, следовательно решений нет. В 3 примере рассмотрим ту же систему но изменим значения правой части. Если проделать ту же процедуру что и в прошлом примере, мы обнаружим, что оба вектора будут совпадать, а значит в качестве решения можно будет взять абсолютно любой вектор.



```
C:\Users\Andrey\source\repos\Gauss Method\x64\Debug\Gauss Method.exe
Input size of square matrix: 2
Input matrix A (2x2):
1 2
2 4
Input right part b:
3 7

The system looks like:
1 2 | 3
2 4 | 7
Error: No solutions
Want to input new right part? (Y/N) _
```

Рис. 2 Выброс исключения: “No solutions”



```
C:\Users\Andrey\source\repos\Gauss Method\x64\Debug\Gauss Method.exe
Input size of square matrix: 2
Input matrix A (2x2):
1 2
2 4
Input right part b:
3 6

The system looks like:
1 2 | 3
2 4 | 6
Error: Infinite number of solutions
Want to input new right part? (Y/N)
```


Рис. 3 Выброс исключения “Infinite number of solutions”

Заключение

В ходе выполнения лабораторной работы я написал программу, с помощью которой можно методом Гаусса-Жордана решить Систему Линейных Алгебраических уравнений, путём элементарных преобразований. Были реализованы шаблонные классы Вектора, Матрицы и СЛАУ. Также была протестирована механика выброса исключений и их дальнейшей обработки. Во всех тестах было получен верный ответ, что позволяет заключить, что программа работает корректно и возможно её дальнейшее использование, для решения подобных задач.

Литература

1. Керниган Б., Ритчи Д., Фьюэр А. Язык программирования СИ //М.: Финансы и статистика. – 1992.
2. Кнут Д. Э. Искусство программирования: Сортировка и поиск. – Издательский дом Вильямс, 2000. – Т. 3.

Приложение

1. Функция main()

```
int main()
{
    int n;
    cout << "Input size of square matrix: ";
    cin >> n;

    SLAU<double> system(n);

    cout << "Input matrix A (" << n << "x" << n << "):" << endl;
    system.input_m();

    char repeat;
    do
    {
        MyVector<double> b(n);
        cout << "Input right part b:" << endl;
        b.input();

        cout << "\n The system looks like:" << endl;
        system.printWhole(b);

        try
        {
            MyVector<double> x = system.Gauss_Jordan(b);
            cout << "Solutions are: " << endl;
            x.print();
            cout << endl;
        }
        catch (const runtime_error& e)
        {
            cout << "Error: " << e.what() << endl;
        }

        cout << "Want to input new right part? (Y/N) ";
        cin >> repeat;
    } while (repeat == 'Y');
```